Department of Computer Science and Engineering

Database Management Systems

Lab 09 | Advanced SQL (CTE and CASE) and Key-Value Database (Redis)

Task 1: Smart Hostel Management System | Task 2: PromptPal - An AI Chat Platform

Student Copy

Tasks in this lab

Task	Topic	Case study	Questions	Submission files
Task 1	Advanced SQL: CTE and CASE (MySQL)	Smart Hostel Management System	Q1 - Q6	<SRN>_CTE_CASE_Lab9.sql, <SRN>_CTE_CASE_Lab9.pdf
Task 2	Key-Value Database: Redis (redis-cli)	PromptPal - An AI Chat Platform	Q1 - Q10	<SRN>_RedisLab.pdf, <SRN>_RedisLab.txt

Task 1: CTE and CASE Lab

Case study: Smart Hostel Management System | **Lecture reference:** Lecture 33 - Common Table Expressions (CTE) and CASE

Objective

To solve demanding data-retrieval problems on the Smart Hostel Management System database with Common Table Expressions (including recursive CTEs) and CASE expressions in MySQL, producing exactly specified, verifiable results.

Outcome

By completing this task, students will be able to:

- Decompose a multi-stage computation into named intermediate results and combine them correctly.
- Compare a value against aggregates of other aggregates without losing rows or duplicating them.
- Traverse a self-referencing hierarchy to any depth with a correct starting point and a guaranteed stopping condition.
- Generate a sequence of dates and carry a running state from one step to the next.
- Write conditional expressions whose branch order, boundaries and NULL behaviour exactly match a written business rule.
- Preserve ties, zero-valued groups and date boundaries in every result.

Case Study: Smart Hostel Management System

The Smart Hostel Management System (Lab 3 / Lab 5 schema) is loaded as the isolated database HOSTEL_LAB9. Every staff member works in one hostel and may report to one supervisor; a room belongs to one hostel and its beds are BED rows; ALLOCATE records each student-room stay; a payment is made by one student and a receipt is issued for at most one payment; USES records which student uses which facility. The table below shows what is stored, in which table, and under which key.

What we store	Table	Key
Hostels	HOSTEL	Hostel_ID
Staff and their supervisors	STAFF	Staff_ID (Supervisor_ID refers to STAFF)
Rooms	ROOM	(Hostel_ID, Room_Number)
Beds and their availability	BED	(Hostel_ID, Room_Number, Bed_Number)
Students	STUDENT	Student_ID
Student phone numbers	STUDENT_PHONE	(Student_ID, Phone_Number)
Facilities	FACILITY	Facility_ID
Facility use	USES	(Student_ID, Facility_ID)
Room stays	ALLOCATE	(Hostel_ID, Room_Number, Student_ID)
Payments	PAYMENT	Payment_ID
Receipts	RECEIPT	Receipt_ID (Payment_ID unique)

Full column definitions, the Lab 9 supplementary records and every business rule are given in the Lab 9 - Database and Business Rules Reference, On Windows, MySQL stores table names in lower case; SQL may use any letter case.

Software Requirements

Item	Requirement
Database server	MySQL Community Server 8.0 or later (the answer key was verified on 8.4.9)
Client	mysql command-line client (MySQL Command Line Client on Windows)
Files	Lab_9_Dataset.sql and

Before You Begin

1. Before executing any command, set the prompt in the mysql command-line client to show your SRN and name, the host and the current database:

```
prompt <SRN>_<NAME>@\h [\d]>\_
```

2. Create the lab database (this drops and re-creates only HOSTEL_LAB9) and verify it:

```
source <path>/Lab_9_Dataset.sql
USE HOSTEL_LAB9;
SHOW TABLES;
```

3. Read the Lab 9 - Database and Business Rules Reference (. It is part of every question.
4. Use only the SQL taught up to Lecture 33: joins, grouping and aggregate functions, nested and correlated subqueries, WITH (common table expressions, including WITH RECURSIVE) and CASE expressions. Window functions, user variables, stored programs, views and temporary tables are not permitted. Each answer must be one read-only statement.
5. Do not modify the dataset. Your query must produce the correct rows on this dataset and must remain correct on any other data that obeys the same reference rules.

Deliverables

- An SQL file containing the six answer queries, each preceded by a comment naming the question: <SRN>_CTE_CASE_Lab9.sql
- A PDF of screenshots from the mysql command-line client, with the prompt set as in Instruction 1, showing each query and its complete output: <SRN>_CTE_CASE_Lab9.pdf

Questions to be executed

Answer all six questions. Each answer is a single executable MySQL statement. Question Qn is governed by Part B and by section Cn of the Lab 9 - Database and Business Rules Reference, which fixes every definition, boundary, tie rule, output column and sort order.

- 6 questions in total (Q1 - Q6); every question is written on the Smart Hostel Management System case study above.

Task 1 Questions, Q1 - Q6

Q1. Resident payment leaders

Compare every hostel's average resident payment total. Keep hostels beating the campus benchmark average. Name each such hostel's largest resident payers.

Q2. Superior salary inversions

Some staff earn more than their superiors. Superiors include supervisors at every level above. Summarise each affected staff member's salary inversions.

Q3. Room occupancy reconciliation

Reconcile every room's bed flags against residents. Each room receives exactly one reconciliation status. Report each hostel's status tallies and corrections.

Q4. Staff salary revision

Apply the published salary revision to everyone. Enforce every supervisor cap exactly as defined. Compare each hostel's current and revised payroll.

Q5. Facility engagement profile

Profile each current resident's home facility engagement. Every resident belongs to one engagement class. Report each hostel's most common engagement classes.

Q6. Daily fee ledger

Simulate every resident's daily hostel fee ledger. Late fees compound under the published rules. Report each penalised student's fees and balance.

Task 1 Submission rules

- Both files are to be submitted: <SRN>_CTE_CASE_Lab9.sql and <SRN>_CTE_CASE_Lab9.pdf.
- Screenshots must be taken from the mysql command-line client with the prompt set as in Instruction 1, and must show each query together with its complete output.
- Assessment is entirely on executable SQL: correctness on the supplied dataset, correctness on additional rule-conforming test data, correctness of the implemented logic, termination and efficiency, and MySQL validity.
- Any correct SQL that follows Instruction 4 and returns the specified output is accepted; it need not match a model answer.

Task 2: Redis Lab

Case study: PromptPal - An AI Chat Platform | **Topic:** Key-Value Database: Redis

Objective

To understand and practise the five core Redis data structures — Strings, Lists, Hashes, Sets and Sorted Sets — using redis-cli on a real-world scenario, so that the fast-changing parts of an AI chat application (quotas, chat history, profiles, memberships and rankings) are stored and queried with the most suitable Redis command.

Outcome

By completing this task, students will learn to:

- Choose the Redis data type that fits a stated requirement.
- Create and read many keys at once with MSET and MGET, and make a key expire with SET … EX.
- Change a string in place with APPEND and INCRBY, including with a negative number.
- Add, view, remove and trim list elements at either end with RPUSH, LPUSH, LRANGE, LLEN, LPOP, RPOP and LTRIM.
- Store and update field-value pairs with HSET, HGET, HGETALL, HINCRBY, HLEN, HDEL and HEXISTS.
- Keep unique members with SADD, SMEMBERS, SISMEMBER, SCARD and SREM.
- Rank members with ZADD, ZCARD, ZREVRANGE, ZINCRBY, ZRANK, ZREVRANK and ZREM.
- Read the integer and (nil) replies of Redis and recognise the WRONGTYPE error.

Case Study: PromptPal

You are a backend engineer at PromptPal, a popular AI chatbot app. Millions of people send messages every day, so Redis handles the parts that must be fast. The table below shows what is stored, in which Redis type, and under which key.

What we store	Redis type	Key name
Daily message quota, OTPs, status	String	quota:u1, otp:u1, status:u1
Chat history	List	conv:101
User profile	Hash	user:1
Beta-program members	Set	beta:gpt-next
Token-usage leaderboard	Sorted Set	leaderboard:tokens

Key names are case-sensitive. u1, u2, … are user IDs; ava, ben, cara and dev are leaderboard member names.

Before You Begin

- Start the Redis server and open the client with redis-cli. Check the connection with PING; the reply must be PONG.
- Make your SRN visible: the first command of every session is ECHO "SRN", with your own SRN in place of SRN. Your SRN must be clearly visible inside every screenshot.
- Start every question on an empty database by running FLUSHDB. Within a question, run the numbered parts in the order given.
- Use the key names, field names, member names and values given in the questions exactly as written.
- Execute every command in redis-cli and verify the reply before recording the output. For each question, record the commands you wrote together with a screenshot of the commands and their replies.
- Write your commands in a text file after executing them. Put a comment line such as # Q1 before the commands of each question.

Deliverables

- A single PDF containing, in order, each of the 10 questions with the commands you wrote and a screenshot of its output, plus the text file with the complete list of commands. Label each screenshot with its question number.
- File names: <SRN>_RedisLab.pdf and <SRN>_RedisLab.txt

Questions to be executed

- Easy level — Questions 1 – 2
- Medium level — Questions 3 – 6
- Hard level — Questions 7 – 10

- 10 questions in total; every question is written on the PromptPal case study above.

Note on Redis and its Data Types

Redis is an in-memory key-value database. Every value is stored under a unique key, and the type of the value is fixed by the command that first creates the key. After that, only the commands of that type work on the key; using a command of another type returns a WRONGTYPE error. The table below is the basis on which the correct type is chosen in each question of this task.

Type	What it stores	Order and duplicates	Typical use here
String	One value: text or a number	One value per key	Quota, OTP, status
List	A sequence of strings	Keeps insertion order; duplicates allowed	Chat history, ticket queue
Hash	Field-value pairs under one key	Fields are unique; setting a field again overwrites it	User profile
Set	A collection of strings	No order; no duplicates	Beta members, RSVP list
Sorted Set	Unique members, each with a numeric score	Ordered by score; ties ordered alphabetically	Leaderboard

Command reference

Type	Command	Purpose
String	SET key value [EX s]	Store a value; EX makes the key expire after s seconds
	GET key	Read a value
	MSET k1 v1 k2 v2	Set many keys in one command
	MGET k1 k2	Read many keys; a missing key shows (nil)
	APPEND key text	Add text at the end; returns the new length
	INCRBY key n	Add n, which can be negative, to a numeric string; returns the new value
List	RPUSH / LPUSH key v1 v2	Add at the end / at the front; returns the new length
	LRANGE key start stop	Read a range; -1 is the last element
	LLEN key	Number of elements
	LPOP / RPOP key	Remove and return the first / last element
	LTRIM key start stop	Keep only the elements in the range
Hash	HSET key f1 v1 f2 v2	Set fields; returns the number of new fields
	HGET key field	Read one field
	HGETALL key	Read all fields and values
	HINCRBY key field n	Add n to a numeric field; a missing field starts at 0
	HLEN key	Number of fields
	HDEL key field	Delete a field; returns 1 if deleted, else 0
	HEXISTS key field	1 if the field exists, else 0
Set	SADD key m1 m2	Add members; returns the number of new members
	SMEMBERS key	Show all members
	SISMEMBER key m	1 if m is a member, else 0
	SCARD key	Number of members
	SREM key m	Remove a member; returns 1 if removed, else 0
Sorted Set	ZADD key score member	Add members with scores, score first; returns the number of new members
	ZCARD key	Number of members
	ZREVRANGE key start stop [WITHSCORES]	Members from highest to lowest score
	ZINCRBY key n member	Add n to a score; returns the new score
	ZRANK key member	0-based position from the lowest score
	ZREVRANK key member	0-based position from the highest score
	ZREM key member	Remove a member

Points to remember

- A reply of (integer) 0 from SADD, HSET, ZADD, HDEL or SREM is not an error: it means nothing new was added or removed.
- SADD, HSET and ZADD return the number of new members or fields only; an existing one that is updated is not counted.
- A negative index counts from the end: -1 is the last element, -5 is the fifth from the last.

- In ZADD the score comes first and the member second.
- A key can hold only one type. Running a List command on a Hash key returns WRONGTYPE.
- A missing key returns (nil) when read, and a missing hash field is created from 0 by HINCRBY.

Easy Level, Q1 - Q2

Strings and Lists are the simplest Redis types. Each question below uses one of them on an empty database. Record the commands, the replies and a screenshot.

Q1. Daily quota

At midnight PromptPal resets every user's daily message allowance. User u1 is allowed 20 messages and user u2 is allowed 50. Each allowance is a single plain value stored under its own key.

1. Using a single command, store both allowances at once: 20 under the key quota:u1 and 50 under the key quota:u2.
2. Using a single command, read both allowances back so that the reply lists the value of quota:u1 first and the value of quota:u2 second.

Q2. Start a chat

Conversation conv:101 begins with the user's message "Hello", followed by the assistant's reply "Hi! How can I help?". The conversation is an ordered sequence of messages in which duplicates are allowed.

1. Using one command, add both messages to the END of conv:101 in the order they were sent , with "Hello" first, so that the oldest message is at the first position and the newest is at the last position.
2. Display every message in the conversation, from the first position to the last.
3. The app must place "SYSTEM: be concise" at the very top of the conversation - before "Hello" - without changing the order of the messages already stored. Place it there.

Medium Level, Q3 - Q6

Medium questions use several commands on one key and ask you to read the replies carefully. Run FLUSHDB before each question.

Q3. User profile

Meera's profile is kept under the single key user:1 as a group of named fields, each with its own value.

1. Create user:1 in one command with the fields name = "Meera", plan = "free" and credits = "10". Note the number Redis replies with.
2. Read only the value of her plan field.
3. Read every field of her profile together with its value.
4. Meera purchases 50 additional credits. Increase only the credits field by 50 so that Redis performs the addition and replies with her new balance, without re-entering or rewriting the other fields.
5. Report how many fields her profile contains.

Q4. OTP and quota

PromptPal keeps short-lived login codes, message allowances and presence status as single plain values, one per key.

1. Store the one-time password "483920" under otp:u1 so that Redis removes it automatically 120 seconds later. Use the general-purpose command that stores a single value, with its expiry-in-seconds option, so that the value and the expiry are applied together in one command.
2. Store "20" under quota:u1. The user then sends 3 messages: reduce the stored allowance by 3 so that Redis performs the subtraction itself and replies with the new value. Do not overwrite the key with a number you worked out yourself. Express the reduction as adding the negative amount -3 to the stored number.
3. Store "Online" under status:u1. Then attach the text " (busy)", which begins with a space, to the end of that value without rewriting the whole value. What number does Redis reply with, and what does it represent?
4. Using a single command, read otp:u1, quota:u1 and quota:u99, in that order. quota:u99 was never created. What does Redis show for it?

Q5. Beta program

The beta programme for the next model keeps its participants under beta:gpt-next. A participant can appear at most once and the order of participants does not matter.

1. Enrol u1, u2 and u3 in beta:gpt-next using one command.
2. Display every participant currently enrolled.
3. Check whether u3 is enrolled. Redis should answer 1 for yes or 0 for no.
4. u2, who is already enrolled, and a new user u4 apply together. Enrol both with one command. What number does Redis return, and why?
5. Report the total number of participants.
6. u4 leaves the programme. Remove them.

Q6. Leaderboard

The token-usage leaderboard leaderboard:tokens holds each player once, together with a numeric score, and is always kept ordered by that score.

1. Record the token counts ava = 500, ben = 800 and cara = 300 on leaderboard:tokens.
2. Report how many players the leaderboard holds.
3. Display only the player names, for every position from the first to the last, ordered from the highest score to the lowest.
4. Cara uses 700 more tokens. Increase her existing score by 700 so that Redis performs the addition and replies with her new total.
5. Display the names together with their scores, highest first. Who is #1 now?

Hard Level, Q7 - Q10

Hard questions need many steps in sequence and ask you to predict or explain what Redis returns. Run FLUSHDB before each question.

Q7. Support ticket queue

The key tickets is PromptPal's support queue. Agents always serve the ticket at the front, which is the first position, and normal tickets join at the back.

Use the list tickets.

1. Three normal tickets "T1", "T2" and "T3" arrive in that order. Place them at the back of the queue using one command, so that T1 is ahead of T2 and T2 is ahead of T3.
2. An urgent ticket "URGENT-1" arrives. It must become the next ticket served, ahead of every waiting ticket, while the relative order of the tickets already waiting is preserved.
3. Display all tickets from the front of the queue to the back.
4. An agent takes the ticket at the front. Remove it so that Redis hands it back. What does Redis return?
5. The customer who raised the ticket at the back cancels it. Remove the ticket at the back so that Redis hands it back. What does Redis return?
6. Two new normal tickets "T4" and "T5" arrive in that order. Place them at the back.
7. Capacity is cut: with a single command keep only the first two tickets, at positions 0 and 1 from the front, and discard all others. Then display the queue.

Q8. Updating a profile

Ravi's profile is kept under the single key user:2 as a group of named fields.

Use the key user:2.

1. Create user:2 in one command with name = "Ravi", plan = "pro" and credits = "100".
2. Ravi used 30 credits. Decrease the credits field by 30 so that Redis performs the arithmetic on the stored field and replies with the new balance.
3. Start a counter field called messages at 1, using one command that increases a numeric field by 1, even though the messages field does not exist yet.

4. Ravi wants his plan field removed. Delete only that field; the rest of the profile must remain.
5. Check whether the plan field is still present in the profile. Redis should answer 1 if it is present and 0 if it is absent.
6. Report how many fields remain in the profile.
7. Display every remaining field together with its value.

Q9. Leaderboard race

leaderboard:tokens holds each player once with a numeric token score and is kept ordered by that score.

Use leaderboard:tokens.

1. Record ava = 500, ben = 800, cara = 300 and dev = 600 on leaderboard:tokens using one command.
2. Report the number of players.
3. Display the player names only, for every position from the first to the last, ordered from the highest score to the lowest.
4. Ava earns 400 more tokens. Increase her score by 400 so that Redis replies with her new total.
5. Determine Ava's zero-based position when the players are ordered from the highest score to the lowest, where the top player is position 0.
6. Determine Cara's zero-based position when the players are ordered from the lowest score to the highest, where the bottom player is position 0.
7. ben was cheating. Remove him from the leaderboard, then display the remaining names with their scores, highest first.

Q10. Pick the right type

New user u5 joins PromptPal. For each need in parts 1-5, decide which Redis data type matches the behaviour described, name that type, and write the command. Parts 6 and 7 ask you to predict and explain a reply.

1. u5's one-time password "7781" must be held as one single value under otp:u5 and must disappear automatically 60 seconds after it is stored.
2. u5's first chat message "Hi" must be kept under conv:5 in a structure that preserves the order in which messages arrive, with later messages placed after it.
3. u5's profile under user:5 must group several named attributes under one key: name = "Neha" and plan = "free".
4. u5 must join beta:gpt-next, the beta programme's participant collection, in which the same participant can never appear twice and order does not matter.
5. u5 must appear in leaderboard:tokens with 250 tokens, where participants are kept ordered by a numeric score.
6. Run the command from part 4 again, exactly as before. What does Redis return, and why?
7. By mistake, a developer's code treats user:5 as if it were a message history like conv:5 and tries to place "hello" as a new last message in it, using the same kind of operation you used in part 2. Run that operation. What happens, and why?

Task 2 Submission rules

- Both files are to be uploaded. A submission that is missing either of them is treated as incomplete.
- Your SRN must be visible in every screenshot. A screenshot in which your SRN cannot be read will not be evaluated.
- Each screenshot must display the commands together with what Redis replied. A screenshot that displays only the text of a command is not evaluated.
- The key names, field names, member names, values and amounts given in the questions are part of the specification and are evaluated.
- Run FLUSHDB before each question so that the replies match the question.